

Experiment – 2

Name: Renulasya Reddy

UID: 24BAI70210

Branch: CSE – AIML (CS221)

Section/Group: 24AIT-NTTP-2[A]

Semester: 5th

Date of Performance: 27/07/2026

Subject: Full Stack - II

Subject Code: 24CSP-337

EXPERIMENT – 3.1

1. **AIM:** To design and implement a secure authentication system using JWT for user login and session management.

2. **OBJECTIVE:**

1. To understand authentication in web applications.
2. To implement token-based authentication using JWT.
3. To manage user sessions using a stateless approach.
4. To securely store and validate JWT tokens.

3. **SOFTWARE REQUIREMENTS:**

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. JWT library

4. **THEORY**

Authentication is the process of verifying a user's identity before allowing access to an application. Modern web applications commonly use **JWT (JSON Web Token)** for authentication because it is secure and stateless.

A JWT consists of three parts:

1. **Header** – Contains the token type and signing algorithm.
2. **Payload** – Contains user information (claims).
3. **Signature** – Verifies that the token has not been modified.

When a user logs in, the server validates the credentials and generates a JWT. The client stores the token (usually in Local Storage or Session Storage) and sends it with every request. The server verifies the token before allowing access to protected resources. Since the server does not maintain session data, JWT-based authentication is called **stateless authentication**.

5. IMPLEMENTATION/PROCEDURE

1. create a React project.
2. Design a login form.
3. Validate user credentials using mock/static data.
4. Generate a simulated JWT token.
5. Store the token in Local Storage.
6. Decode the token to retrieve user information.
7. Display the authenticated user details.
8. Implement Logout functionality.
9. Execute the application using npm start.
10. Verify successful login and logout.

6. CODE

Login.js

```
import React, { useState, useEffect } from "react";
function Login() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [user, setUser] = useState(null);
  useEffect(() => {
    const token = localStorage.getItem("token");
    if(token){
      try{
        const payload =
JSON.parse(atob(token.split(".")[1]));
        setUser(payload);
      }
      catch{
        localStorage.removeItem("token");
      }
    }
  }, []);
  const handleLogin = () => {
    if(username==="admin" && password==="1234"){
      const payload={
        username:"admin",
        role:"Student"
      };
      const
token="header."+btoa(JSON.stringify(payload))+".signature";
      localStorage.setItem("token",token);
      setUser(payload);
    }
    else{
      alert("Invalid Username or Password");
    }
  };
  const handleLogout = () => {
    localStorage.removeItem("token");
```

```

    setUser(null);
    setUsername("");
    setPassword("");
  };
  return (
    <div className="container">
      <h1>🔒 JWT Authentication</h1>
      {!user ? (
        <div className="login-box">
          <input
            type="text"
            placeholder="Username"
            value={username}
            onChange={(e)=>setUsername(e.target.value)}
          />
          <input
            type="password"
            placeholder="Password"
            value={password}
            onChange={(e)=>setPassword(e.target.value)}
          />
          <button onClick={handleLogin}>
            Login
          </button>
          <p className="note">
            Username : admin <br/>
            Password : 1234
          </p>
        </div>
      ) : (
        <div className="welcome">
          <h2>Welcome {user.username}</h2>
          <h3>Role : {user.role}</h3>
          <p>✅ Token Generated Successfully</p>
          <button onClick={handleLogout}>
            Logout
          </button>
        </div>
      )}
    </div>
  );
}
export default Login;

```

App.css

```

body{

margin:0;

font-family:Arial;

background:linear-gradient(to
right,#6dd5ed,#2193b0);

}

```



```
.container{  
  
width:450px;  
  
margin:60px auto;  
  
background:white;  
  
padding:30px;  
  
border-radius:12px;  
  
box-shadow:0 0 15px gray;  
  
text-align:center;  
  
}  
  
h1 {  
  
color:#4b0082;  
  
}  
  
input{  
  
width:90%;  
  
padding:12px;  
  
margin:10px;  
  
font-size:16px;  
  
border-radius:5px;  
  
border:1px solid gray;  
  
}  
  
button{  
  
padding:12px 25px;  
  
background:#6a1b9a;  
  
color:white;  
  
border:none;  
  
border-radius:5px;  
  
font-size:16px;  
  
cursor:pointer;  
  
}  
  
button:hover{
```



```
background:#4a148c;
```

```
}
```

```
.note{
```

```
margin-top:20px;
```

```
color:gray;
```

```
}
```

```
.welcome{
```

```
font-size:20px;
```

```
}
```

```
h2{
```

```
color:green;
```

```
}
```

App.js

```
import Login from "../components/Login";
```

```
import "../App.css";
```

```
function App() {
```

```
  return (
```

```
    <div>
```

```
      <Login />
```

```
    </div>
```

```
  );
```

```
}
```

```
export default App;
```

index.js

```
import React from "react";
```

```
import ReactDOM from "react-dom/client";
```

```
import "../App.css";
```

```
import App from "../App";
```

```
const root =
```

```
ReactDOM.createRoot(document.getElementById("root"));
```

```
root.render(
```

```
<React.StrictMode>
```

```
<App />
```

```
</React.StrictMode>
```

```
);
```

7. OUTPUT

The application successfully authenticates users using a simulated JWT token. After successful login, the token is stored in Local Storage and user details are displayed. The session remains active until the user logs out, demonstrating stateless authentication.

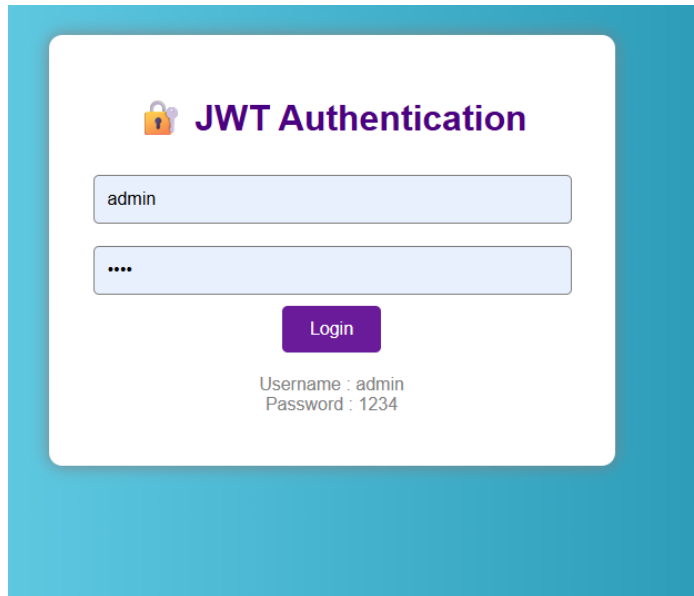
A screenshot of a web application's login form. The form is titled 'JWT Authentication' with a lock icon. It contains two input fields: the first is labeled 'admin' and the second is masked with four dots. Below the fields is a purple 'Login' button. At the bottom, it displays 'Username : admin' and 'Password : 1234'.

Fig 3.1: This is the first screenshot showing the username, password fields, and Login button.

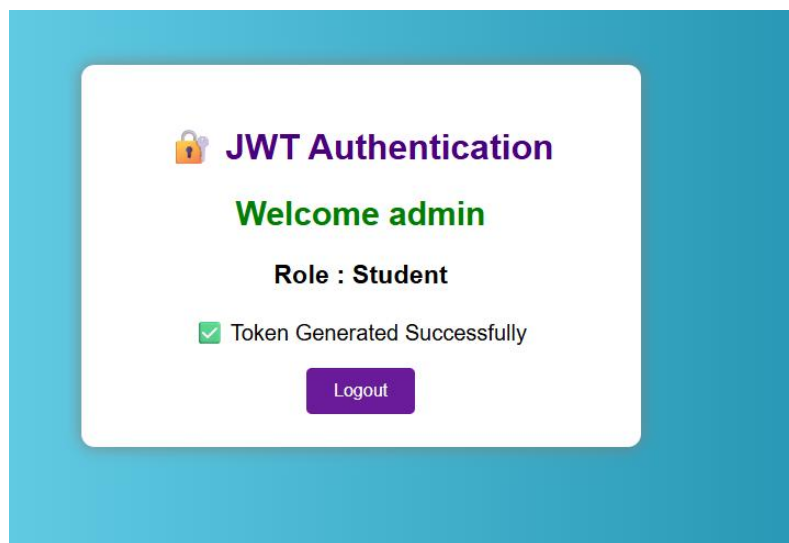
A screenshot of the web application's welcome screen after a successful login. The title 'JWT Authentication' remains. Below it, 'Welcome admin' is displayed in green. Underneath, 'Role : Student' is shown. A green checkmark icon is followed by the text 'Token Generated Successfully'. At the bottom, there is a purple 'Logout' button.

Fig 3.1: This is the second screenshot showing "Welcome admin", role, token generated, and Logout button.

8. LEARNING OUTCOME

- Understand JWT-based authentication.
- Learned stateless authentication.
- Generated and stored JWT tokens.
- Managed user sessions using Local Storage.
- implemented login and logout functionality.
- Gained knowledge of secure authentication mechanisms.

EXPERIMENT – 3.2

1. AIM: To implement Role-Based Access Control (RBAC) and secure application routes based on user permissions.

2. OBJECTIVE:

1. To understand authorization mechanisms.
2. To implement Role-Based Access Control (RBAC).
3. To protect application routes based on user roles.
4. To dynamically render UI elements according to user permissions.

3. SOFTWARE REQUIREMENTS:

1. Node.js (LTS Version)
2. NPM (Node Package Manager)
3. Visual Studio Code
4. Google Chrome / Microsoft Edge
5. React.js
6. React Router DOM
7. Command Prompt / Terminal

4. THEORY

Authorization determines what actions an authenticated user is allowed to perform. **Role-Based Access Control (RBAC)** is a security model where permissions are assigned based on user roles instead of individual users.

Common roles include:

- **Admin** – Full access.
- **Editor** – Can create and edit content.
- **Viewer** – Can only view content.

React Router is used to protect routes so that only users with the required role can access specific pages. If a user attempts to access an unauthorized page, they are redirected to an **Unauthorized** page. RBAC improves application security, simplifies permission management, and ensures controlled access to sensitive features.

5. IMPLEMENTATION/PROCEDURE

1. Create a React project.
2. Install React Router DOM.
3. Create Login, Admin, Editor, Viewer, and Unauthorized pages.
4. Define user roles (Admin, Editor, Viewer).
5. Store the selected role in Local Storage.
6. Create a ProtectedRoute component.
7. Verify the user's role before allowing route access.
8. Redirect unauthorized users to the Unauthorized page.
9. Display different dashboards based on the selected role.
10. Run the application and verify role-based access.

6. CODE

ProtectedRoute.js

```
import { Navigate } from "react-router-dom";

function ProtectedRoute({ children, role }) {

  const userRole = localStorage.getItem("role");

  if (userRole === role) {

    return children;

  }

  return <Navigate to="/unauthorized" />;

}

export default ProtectedRoute;
```

Admin.js

```
import { useNavigate } from "react-router-dom";

function Admin() {

  const navigate = useNavigate();

  const logout = () => {
```

```
localStorage.removeItem("role");

navigate("/");

};

return (

  <div className="container">

    <h1> 👤 Admin Dashboard</h1>

    <h3>Welcome Admin</h3>

    <p>You have full access.</p>

    <button onClick={logout}>
      Logout
    </button>

  </div>

);

}
```

export default Admin;

Editor.js

```
import { useNavigate } from "react-router-dom";

function Editor() {

  const navigate = useNavigate();

  const logout = () => {

    localStorage.removeItem("role");

    navigate("/");

  };

  return (

    <div className="container">
```

```
<h1>Editor Dashboard</h1>
```

```
<h3>Welcome Editor</h3>
```

```
<p>You can edit content.</p>
```

```
<button onClick={logout}>
```

```
  Logout
```

```
</button>
```

```
</div>
```

```
);
```

```
}
```

```
export default Editor;
```

Login.js

```
import { useState } from "react";
```

```
import { useNavigate } from "react-router-dom";
```

```
function Login() {
```

```
  const [role, setRole] = useState("");
```

```
  const navigate = useNavigate();
```

```
  const handleLogin = () => {
```

```
    if (role === "") {
```

```
      alert("Please Select a Role");
```

```
      return;
```

```
    }
```

```
    localStorage.setItem("role", role);
```

```
    if (role === "Admin") {
```

```
      navigate("/admin");
```

```
    }
```

```
    else if (role === "Editor") {
```

```
    navigate("/editor");

  }

  else {

    navigate("/viewer");

  }

};

return (

  <div className="container">

    <h1>Role Based Access Control</h1>

    <h3>Select Your Role</h3>

    <select
      value={role}
      onChange={(e) => setRole(e.target.value)}
    >

      <option value="">Select Role</option>

      <option value="Admin">Admin</option>

      <option value="Editor">Editor</option>

      <option value="Viewer">Viewer</option>

    </select>

    <br /><br />

    <button onClick={handleLogin}>
      Login
    </button>

  </div>

);

}

export default Login;
```

Unauthorized.js

```
import { Link } from "react-router-dom";

function Unauthorized() {

  return (

    <div className="container">

      <h1>✖ Unauthorized Access</h1>

      <p>You don't have permission to access this page.</p>

      <Link to="/">
        Go Back
      </Link>

    </div>

  );

}

export default Unauthorized;
```

Viewer.js

```
import { useNavigate } from "react-router-dom";

function Viewer() {

  const navigate = useNavigate();

  const logout = () => {

    localStorage.removeItem("role");

    navigate("/");

  };

};
```

```
return (  
  
  <div className="container">  
  
    <h1>👁 Viewer Dashboard</h1>  
  
    <h3>Welcome Viewer</h3>  
  
    <p>You can only view content.</p>  
  
    <button onClick={logout}>  
      Logout  
    </button>  
  
  </div>  
  
);  
  
}
```

```
export default Viewer;
```

App.css

```
body{  
  margin:0;  
  font-family:Arial;  
  background:linear-gradient(to  
    right,#74ebd5,#ACB6E5);  
}  
  
.container{  
  width:450px;  
  margin:70px auto;  
  padding:30px;  
  background:white;  
  text-align:center;  
  border-radius:10px;  
  box-shadow:0 0 10px gray;  
}  
  
h1 {
```

```
color:#4b0082;  
}
```

```
select{  
width:90%;  
padding:10px;  
font-size:16px;  
}
```

```
button{  
padding:10px 20px;  
font-size:16px;  
background:#4b0082;  
color:white;  
border:none;  
border-radius:5px;  
cursor:pointer;  
}
```

```
button:hover{  
background:#6a0dad;  
}
```

```
a{  
text-decoration:none;  
font-size:18px;  
}
```

App.js

```
import { Routes, Route } from "react-  
router-dom";  
import Login from "./pages/Login";  
import Admin from "./pages/Admin";  
import Editor from "./pages/Editor";  
import Viewer from "./pages/Viewer";  
import Unauthorized from  
  "./pages/Unauthorized";  
import ProtectedRoute from  
  "./components/ProtectedRoute";
```

```
function App() {
```

```
return (  
  
  <Routes>  
  
    <Route path="/" element={<Login />}  
  />  
  
  <Route  
    path="/admin"  
    element={  
      <ProtectedRoute role="Admin">  
        <Admin />  
      </ProtectedRoute>  
    }  
  />  
  
  <Route  
    path="/editor"  
    element={  
      <ProtectedRoute role="Editor">  
        <Editor />  
      </ProtectedRoute>  
    }  
  />  
  
  <Route  
    path="/viewer"  
    element={  
      <ProtectedRoute role="Viewer">  
        <Viewer />  
      </ProtectedRoute>  
    }  
  />  
  
  <Route path="/unauthorized"  
    element={<Unauthorized />} />  
  
  </Routes>  
  
);
```



```
}
```

```
export default App;
```

Index.css

```
body {  
  margin: 0;  
  font-family: -apple-system,  
  BlinkMacSystemFont, 'Segoe UI',  
  'Roboto', 'Oxygen',  
  'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid  
  Sans', 'Helvetica Neue',  
  sans-serif;  
  -webkit-font-smoothing: antialiased;  
  -moz-osx-font-smoothing: grayscale;  
}
```

```
code {  
  font-family: source-code-pro, Menlo,  
  Monaco, Consolas, 'Courier New',  
  monospace;  
}
```

Index.js

```
import React from "react";  
import ReactDOM from "react-  
  dom/client";  
import { BrowserRouter } from "react-  
  router-dom";  
import "./App.css";  
import App from "./App";  
  
const root =  
  ReactDOM.createRoot(document.getEle  
  mentById("root"));  
  
root.render(  
  <BrowserRouter>  
    <App />  
  </BrowserRouter>  
);
```

7. OUTPUT

The application successfully implements Role-Based Access Control. Users are redirected to different dashboards based on their selected roles. Protected routes prevent unauthorized access, and the interface dynamically changes according to the user's permissions.

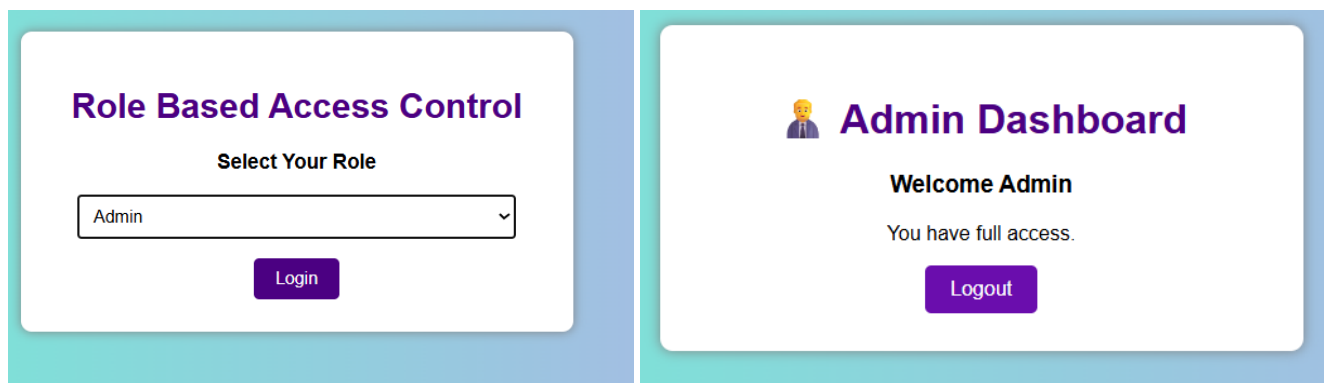


Fig 3.2: Role-Based Access Control – Admin Login and Admin Dashboard

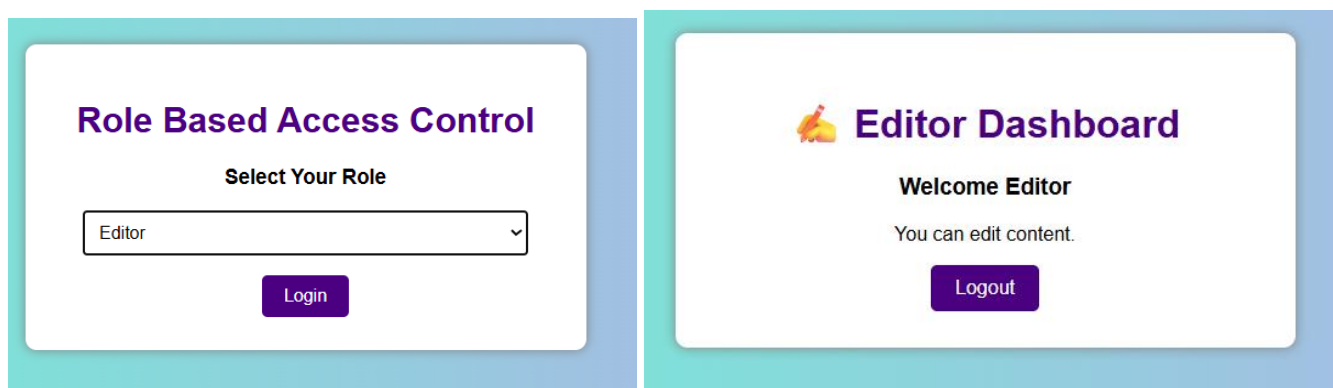


Fig 3.2: Role-Based Access Control – Editor Login and Editor Dashboard

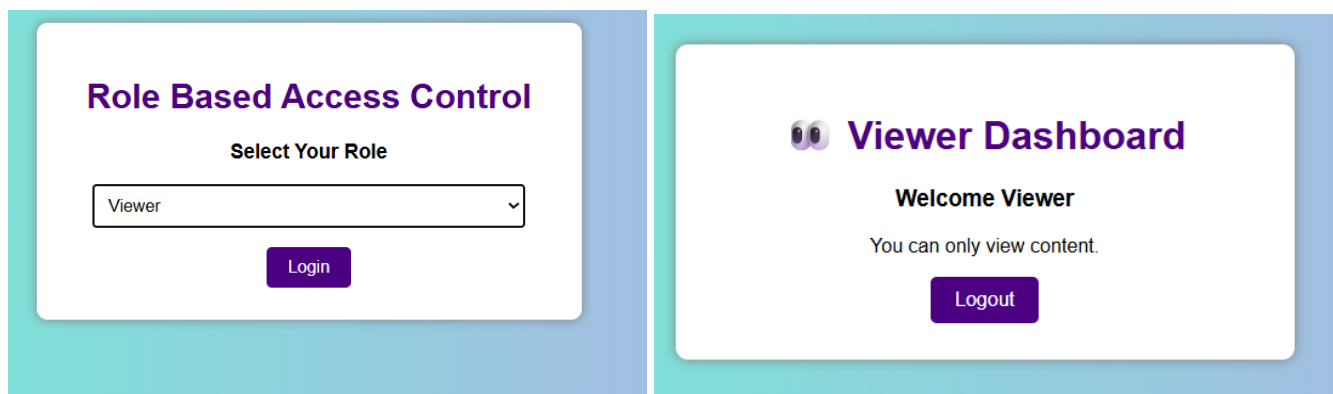


Fig 3.2: Role-Based Access Control – Viewer Login and Viewer Dashboard

8. LEARNING OUTCOME

- Understood authorization and RBAC concepts.
- Implemented protected routes using React Router.
- Restricted access based on user roles.
- Dynamically rendered UI according to permissions.
- Improved application security.
- Learned route protection techniques in React.